Relatório A1- DijkFood

Computação em nuvem

Elisa Soares, Maria Eduarda Mesquita, Mariana Fernandes, Raphaella Roma

Introdução

O DijkFood é um sistema distribuído que simula uma plataforma de entregas sob demanda, inspirado em aplicações reais como iFood e Uber Eats. O objetivo principal do projeto é demonstrar conceitos fundamentais de sistemas distribuídos, como desacoplamento entre serviços, escalabilidade horizontal, tolerância a falhas e consistência de dados, utilizando serviços da AWS.

O sistema é composto por múltiplos microsserviços que interagem entre si por meio de comunicação síncrona (HTTP) e assíncrona (fila de mensagens), permitindo que cada componente opere de forma independente e escalável.

Arquitetura Geral

A arquitetura do sistema segue o padrão de microsserviços orientados a eventos, utilizando comunicação assíncrona baseada em filas para desacoplar os componentes.

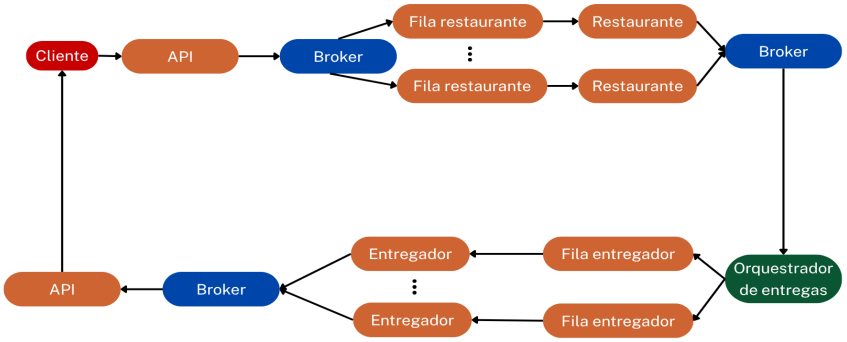
O fluxo do sistema inicia quando o cliente envia uma requisição para a API. Após validação e persistência dos dados, a API publica uma mensagem em um broker de mensagens, que atua como ponto central de distribuição.

A partir desse broker, o pedido é encaminhado para filas específicas de restaurantes, permitindo que múltiplos restaurantes processem pedidos de forma independente e paralela. Cada restaurante consome sua respectiva fila, realiza o preparo do pedido e, ao finalizar, publica uma nova mensagem indicando a conclusão.

Essas mensagens são recebidas por um segundo broker, que as encaminha para um serviço de orquestração de entregas. O orquestrador é responsável por coordenar a logística, distribuindo pedidos para filas de entregadores disponíveis.

Os entregadores consomem essas filas, realizam a simulação da entrega e enviam atualizações de status ao sistema. Essas atualizações passam novamente por um broker e retornam à API, que então disponibiliza as informações atualizadas ao cliente.

Esse fluxo evidencia um pipeline assíncrono completo, no qual diferentes etapas do processamento são desacopladas por meio de múltiplos brokers e filas especializadas. Essa abordagem permite alta escalabilidade, já que restaurantes e entregadores podem ser replicados horizontalmente, além de aumentar a resiliência do sistema, uma vez que falhas em um componente não interrompem o processamento global.



Decisões do Projeto - Uso de mensageria

Foi adotado o uso de mensageria com RabbitMQ como mecanismo de comunicação entre os componentes do sistema, em substituição a abordagens baseadas em polling (loops de verificação contínua de estado).

A principal alternativa considerada foi o uso de chamadas síncronas entre serviços ou loops periódicos para verificar mudanças de estado (polling). Essa abordagem foi descartada por apresentar limitações relevantes:

- **Ineficiência**: polling gera requisições constantes mesmo quando não há novos eventos, desperdiçando recursos computacionais.
- **Maior latência**: há atraso entre a ocorrência de um evento e sua detecção, dependendo do intervalo do loop.
- **Acoplamento elevado**: serviços passam a depender diretamente uns dos outros, dificultando escalabilidade e manutenção.

Em contraste, a mensageria baseada em eventos apresenta vantagens claras:

- **Desacoplamento entre serviços**: produtores e consumidores não precisam se conhecer diretamente.
- **Processamento assíncrono**: permite que cada componente opere no seu próprio ritmo.

- **Escalabilidade horizontal natural**: múltiplos workers podem consumir filas em paralelo.
- **Eficiência**: eventos são processados apenas quando ocorrem, eliminando requisições desnecessárias.
- **Resiliência**: mensagens podem ser persistidas na fila, evitando perda de eventos em caso de falhas.

Essa decisão é fundamental para garantir que o sistema atenda aos requisitos de escalabilidade e isolamento de componentes, especialmente nos cenários de alta carga definidos no trabalho.

Armazenamento (RDS vs NoSQL único)

Uma alternativa seria utilizar apenas um banco NoSQL para todo o sistema. No entanto, isso dificultaria a modelagem de dados relacionais, como pedidos e clientes.

A escolha por uma arquitetura híbrida com RDS e DynamoDB permite combinar consistência forte e transações (RDS) com alta escalabilidade e baixa latência (DynamoDB).

Execução dos serviços (ECS Fargate vs EC2)

Foi considerada a execução dos containers diretamente em instâncias EC2, o que permitiria maior controle e potencial redução de custos.

No entanto, optou-se pelo ECS com Fargate por eliminar a necessidade de gerenciamento de infraestrutura, facilitando o deploy e permitindo escalabilidade automática.

Componentes do Sistema

API (FastAPI)

A API é o ponto de entrada do sistema. Ela expõe endpoints REST que permitem a criação de pedidos, consulta de status e interação com o sistema.

Ao receber uma requisição de criação de pedido, a API realiza validações importantes, como verificar a existência do cliente e do restaurante. Em seguida, executa a criação do pedido de forma transacional, garantindo a atomicidade.

Após a criação bem-sucedida, a API envia uma mensagem para o RabbitMQ, iniciando o processamento assíncrono do pedido.

RabbitMQ (Mensageria)

O RabbitMQ é responsável por desacoplar os serviços. Em vez de os serviços se comunicarem diretamente, eles trocam mensagens através de filas.

Isso permite que:

- serviços funcionem de forma independente
- falhas em um serviço não afetem imediatamente os outros
- o sistema escale de forma mais eficiente

O RabbitMQ foi implantado em uma instância EC2 na AWS.

Routing Service (Dijkstra)

Este serviço é responsável pelo cálculo de rotas entre pontos geográficos, utilizando o algoritmo de Dijkstra.

Ele recebe coordenadas de origem e destino e retorna o caminho ótimo, sendo essencial para a lógica de entrega.

Delivery Service

O Delivery Service consome mensagens da fila e coordena o processo de entrega. Ele interage com o Routing Service para determinar rotas e com outros componentes para atualizar o status do pedido.

Restaurant Worker

Simula o comportamento de restaurantes. Ele consome pedidos da fila, processa o preparo e atualiza o status do pedido.

Courier Worker

Simula entregadores. Cada worker representa um entregador que se move no mapa e envia atualizações constantes de localização.

Essas atualizações são feitas em alta frequência, o que exige um sistema de armazenamento rápido e escalável.

Armazenamento de Dados

Amazon RDS

O banco relacional (PostgreSQL) é utilizado para armazenar dados estruturados, como clientes, restaurantes e pedidos.

Ele garante propriedades ACID, especialmente a atomicidade, que assegura que a criação de pedidos ocorra de forma consistente.

Amazon DynamoDB

O DynamoDB é utilizado para armazenar dados de alta frequência, como a localização dos entregadores.

Por ser um banco NoSQL altamente escalável, ele permite leituras e escritas rápidas, sendo ideal para esse tipo de dado.

Infraestrutura na AWS

Amazon ECS (Fargate)

Os serviços do sistema são executados como containers no ECS utilizando Fargate. Isso elimina a necessidade de gerenciar servidores manualmente.

Cada serviço roda de forma independente e pode ser escalado conforme necessário.

Amazon EC2

O RabbitMQ foi implantado em uma instância EC2, permitindo controle direto sobre a configuração do broker de mensagens.

Deploy Automatizado

Inicialmente, o sistema era executado localmente com Docker Compose. Posteriormente, foi desenvolvido um script `deploy.py` para automatizar o processo de deploy na AWS.

Esse script realiza:

- criação das task definitions no ECS
- criação e atualização dos serviços no ECS
- configuração de infraestrutura necessária (ALB, RDS, DynamoDB e EC2 para o broker)

Dessa forma, o processo de deploy torna-se reproduzível, reduz erros manuais e permite a execução completa do ambiente a partir de um único script, conforme exigido pelo trabalho.

Escalabilidade

O sistema foi projetado para escalar horizontalmente. Isso significa que múltiplas instâncias de um serviço podem ser executadas simultaneamente.

Por exemplo, é possível aumentar o número de courier workers para lidar com maior carga de entregas.

Tolerância a Falhas

A utilização de mensageria garante que, mesmo que um serviço falhe, as mensagens não sejam perdidas e possam ser processadas posteriormente.

Além disso, a execução em ECS permite reinicialização automática de containers em caso de falha.

Simulação de Carga

O sistema inclui um simulador que gera pedidos em diferentes taxas, permitindo testar o comportamento sob carga.

Cenários testados incluem:

- carga normal
- picos de demanda
- eventos de alta intensidade

Modelo de custos

O custo do sistema foi estimado considerando a região us-east-1 da AWS, com base nos serviços utilizados:

- Amazon ECS (Fargate)
- Amazon EC2 (RabbitMQ)
- Amazon RDS
- Amazon DynamoDB
- Amazon Load Balancer

No cenário de **operação normal(10 req/s)**, o sistema utiliza uma quantidade reduzida de instâncias nos serviços ECS, com baixa carga em banco de dados e DynamoDB, resultando em um custo mensal estimado entre **80 e 100 USD**.

No cenário de **pico(50 req/s)**, há aumento do número de instâncias e da taxa de escrita, especialmente no DynamoDB devido às atualizações frequentes de localização dos entregadores, elevando o custo para aproximadamente **250 a 350 USD por mês**.

No cenário de **evento especial (200 req/s)**, o sistema precisa escalar significativamente todos os componentes, principalmente ECS e DynamoDB, que passam a lidar com alto volume de requisições e escritas. Nesse caso, o custo mensal estimado varia entre **700 e 1200 USD**.

O sistema é barato em carga normal, mas caro em alta escala. Sem otimização, o custo é dominado por escrita intensiva no DynamoDB.

Conclusão

O DijkFood demonstra uma arquitetura moderna de sistemas distribuídos, utilizando microsserviços, mensageria e serviços gerenciados na AWS.

O sistema atende aos principais requisitos do projeto, incluindo escalabilidade, consistência, tolerância a falhas e desempenho, sendo uma representação realista de aplicações utilizadas na indústria.

Análise dos Resultados Experimentais

Os experimentos foram realizados no cenário de carga normal (10 pedidos por segundo durante 60 segundos), totalizando 600 requisições de criação de pedidos. O sistema conseguiu receber as requisições previstas, com 416 pedidos aceitos pela API, resultando em um throughput efetivo de aproximadamente 3,8 pedidos por segundo.

Durante a execução, observou-se que parte dos pedidos não completou todo o ciclo de vida dentro da janela de observação do experimento. As latências da operação `POST /orders` apresentaram valores elevados sob carga contínua, indicando que o sistema começa a experimentar contenção quando submetido a um volume constante de requisições. Ainda assim, as operações foram processadas e aceitas em sua maioria, o que demonstra funcionamento correto da API e da integração com o backend.

As operações de consulta *GET/orders/{d* apresentaram comportamento consistente com o estado do sistema durante o experimento, refletindo o fato de que muitos pedidos ainda estavam em processamento no momento das consultas.

Por outro lado, as operações de cadastro *POST/users*, *POST/couriers*, *POST/restaurants* mantiveram desempenho estável, com baixa latência e taxa de sucesso total, evidenciando que os componentes básicos do sistema estão funcionando de forma adequada.

De forma geral, os resultados mostram que o sistema é capaz de operar sob carga e processar pedidos de forma consistente, embora ainda existam oportunidades de otimização na pipeline assíncrona para melhorar a taxa de processamento e reduzir latências em cenários mais exigentes.